

Resolución Trabajo Práctico: Elemento Mayoritario

Algoritmos de Divide y Conquista en Java

Asignatura: Programación III

Paradigma: Divide y Conquista

Lenguaje: Java 17+

1. Implementación en Java

A continuación se presenta el código fuente completo correspondiente a la solución estructurada de la clase `ElementoMayoritario`, que implementa la búsqueda del candidato por reducción de pares y su posterior verificación lineal en $O(n)$.

```
import java.util.Arrays;
import java.util.Optional;

public class ElementoMayoritario {

    /**
     * Método recursivo que reduce el arreglo agrupando elementos en pares
     * según el algoritmo de Divide y Conquista.
     */
    public static Integer buscarCandidato(int[] v, int inicio, int fin) {
        int n = fin - inicio + 1;

        // Casos base
        if (n <= 0) return null;
        if (n == 1) return v[inicio];
        if (n == 2) {
            return (v[inicio] == v[fin]) ? v[inicio] : null;
        }

        // Reducción por pares en la siguiente etapa
        int[] reduccion = new int[n / 2];
        int k = 0;

        for (int i = inicio; i < fin; i += 2) {
            if (v[i] == v[i + 1]) {
                reduccion[k++] = v[i];
            }
        }

        // Llamada recursiva sobre la reducción
        Integer candidato = buscarCandidato(reduccion, 0, k - 1);

        // Manejo de sub-arreglos de tamaño impar
        if (n % 2 != 0) {
            int ultimoElemento = v[fin];

            if (candidato == null) {
                return ultimoElemento;
            }

            int countCandidato = 0;
            int countUltimo = 0;

            for (int i = inicio; i <= fin; i++) {
                if (v[i] == candidato) countCandidato++;
                if (v[i] == ultimoElemento) countUltimo++;
            }

            if (countUltimo > countCandidato) {
```

```

        return ultimoElemento;
    }
}

return candidato;
}

/**
 * Método principal que obtiene el elemento mayoritario y verifica
 * su frecuencia real con una pasada lineal O(n).
 */
public static Optional<Integer> obtenerElementoMayoritario(int[] v) {
    if (v == null || v.length == 0) {
        return Optional.empty();
    }

    // Copia defensiva para evitar side-effects
    int[] copia = Arrays.copyOf(v, v.length);

    // Pasada 1: Obtención del candidato
    Integer candidato = buscarCandidato(copia, 0, copia.length - 1);

    if (candidato == null) {
        return Optional.empty();
    }

    // Pasada 2: Verificación lineal O(n)
    int contador = 0;
    for (int x : v) {
        if (x == candidato) contador++;
    }

    if (contador > v.length / 2) {
        return Optional.of(candidato);
    } else {
        return Optional.empty();
    }
}

public static void main(String[] args) {
    int[][] casosPrueba = {
        {1, 2, 1, 1, 3, 1, 1},
        {1, 2, 3, 4, 5, 6},
        {2, 2, 2, 4, 5, 6}
    };

    for (int i = 0; i < casosPrueba.length; i++) {
        Optional<Integer> res = obtenerElementoMayoritario(casosPrueba[i]);
        System.out.println("Caso " + (i + 1) + ": " +
            (res.isPresent() ? "Elemento Mayoritario = " + res.get() : "No existe elemento mayoritario"));
    }
}

```

2. Casos de Prueba Verificados

Caso	Vector de Entrada	Frecuencia Real / Condición	Resultado Esperado y Obtenido
Caso 1		Frecuencia del 1 = 5/7 ($5 > 3.5$)	Elemento Mayoritario = 1

Caso	Vector de Entrada	Frecuencia Real / Condición	Resultado Esperado y Obtenido
	[1, 2, 1, 1, 3, 1, 1]		
Caso 2	[1, 2, 3, 4, 5, 6]	Ningún elemento se repite > 3 veces	No existe elemento mayoritario
Caso 3	[2, 2, 2, 4, 5, 6]	Frecuencia del 2 = 3/6 (3 no es > 3)	No existe elemento mayoritario

3. Cuestionario de Análisis Teórico

3.1. Ecuación de Recurrencia y Complejidad Temporal

En el método buscarCandidato:

- La etapa de reducción empareja elementos contiguos realizando aproximadamente $n/2$ comparaciones en tiempo lineal $O(n)$.
- Se genera un nuevo sub-arreglo reducido de tamaño a lo sumo $n/2$.
- Se realiza **una única llamada recursiva** sobre el vector de tamaño $n/2$.

La ecuación de recurrencia adopta la forma estándar:

$$T(n) = T(n/2) + \Theta(n)$$

Justificación por Teorema Maestro / Suma Geométrica:

Al desplegar la recurrencia se obtiene la serie:

$$T(n) = c \cdot n + c \cdot (n/2) + c \cdot (n/4) + \dots + c = c \cdot n \sum (1/2)^i \leq 2c \cdot n = \Theta(n)$$

Adicionalmente, el método obtenerElementoMayoritario realiza una segunda pasada de verificación de frecuencia de orden $O(n)$. Por lo tanto, la complejidad temporal global del algoritmo permanece en $\Theta(n)$.

3.2. Efectos Secundarios (Side Effects)

¿Por qué es fundamental operar sobre una copia del arreglo original?

En Java, los arreglos se transmiten mediante referencias a memoria. Modificar directamente el vector recibido como argumento alteraría el estado original reservado por el emisor/cliente.

Consecuencias en memoria:

1. **Corrupción de datos:** Provoca mutación colateral no deseada que destruye la estructura original del vector en la aplicación.
2. **Invalidez de la verificación final:** La etapa de verificación necesita contrastar el candidato contra la frecuencia global en la estructura intacta. Un reordenamiento o filtrado in-place invalidaría el conteo real.